

MINOR ASSIGNMENT-05

Practical Programming with C (CSE 3544)

Publish on: 04-11-2025

Course Outcome: CO₃

Program Outcome: PO₃

Submission on: 11-11-2025

Learning Level: L₄

Problem Statement:

Experiment with pointers and dynamic memory allocation in C.

Assignment Objectives:

To learn how to manipulate arrays using pointers and to learn `malloc()`, `calloc()`, `realloc()` & `free()` to allocate and free dynamic memory.

Answer the followings:

1. Find the output of the code snippet.

```
int main()
{
    int a[][2][4]={5,6,7,8,9,11,12,1};
    printf("%d\n",*(*(*a+0)+1)+2));
    return 0;
}
```

Output▼

2. Find the output and different types of pointer involved in the code snippet;

```
int main()
{
    int *p=NULL;
    p=(int *)malloc(sizeof(int));
    *p=10;
    free(p);
    int *q;
    q=(int *)malloc(sizeof(int));
    *q=15;
    printf("%d  %d\n",*p,*q);
    return 0;
}
```

Output▼

3. State the output of the following program. Assume the address of p is 1000 and q is 2000.

```
#include<stdio.h>
#include<stdlib.h>
void fun(int **q);
int main()
{
    int *p=(int *)malloc(sizeof(int));
    *p=55;
    fun(&p);
    printf("%d %p\n",*p,p);
    return 0;
}
```

```
void fun(int **q){
    int r=20;
    **q=r;
    printf("%p\n",*q);
}
```

Output▼

4. Select the desire output of the following code snippet with reason;

```
int *fun();
int main(void){
    int *ptr;
    ptr=fun();
    printf("%d\n", *ptr);
    return 0;
}

int *fun(){
    int a=10,b=20;
    int sum=0;
    sum=sum+a+b;
    return &sum;
}
```

Output with reason▼

- (A) Unexpected behavior (C) 30
(B) Address of sum (D) None of these

5. Select the desire output of the following code snippet with reason;

```
int *fun();
int main(void)
{
    int *ptr=fun();
    printf("%d\n", *ptr);
    return 0;
}

int *fun(){
    int a=10,b=20,*sum;
    sum=(int *)malloc(
        sizeof(int));
    *sum=a+b;
    return sum;
}
```

Output with reason▼

- (A) Unexpected behavior (C) 30
(B) Address of sum (D) None of these

6. Find the output of the following program.

```
int main(){int *ptr;
ptr=(int *)realloc(NULL,sizeof(int));
*ptr=100;
printf("%d\n", *ptr);
return 0;}
```

Output▼

7. Write the output of the following program.

```
1 int main(){
2     int *ptr;
3     ptr=(int *)calloc(1,sizeof(int));
4     *ptr=100;
5     printf("%d\n", *ptr);
6     ptr=(int *)realloc(ptr, 0);
7     ptr=NULL;
8     printf("%p\n", ptr);
9     return 0;
10 }
```

Output▼

Output at line-4:

Output at line-7:

Line number-6 can be treated as like **free()** to deallocate memory-**Y|N**.

8. Write the output of the code snippet. Also show the stack and heap memory for this application.

```
int main(){
    int i;
    int *p=(int *)malloc(sizeof(int));
    *p=100;
    p=(int *)malloc(5*sizeof(int));
    for(i=0;i<5;i++){
        scanf("%d",p+i); /* 10,20,30,40,50 */
    }
    for(i=0;i<5;i++){
        printf("%d...%d\n",p[i],*(p+i));
    }
    return 0;
}
```

Output▼

9. Write the output of the code snippet. Also show the stack and heap memory for this application.

```
int main() {
    int i,*p,*rp;
    p=(int *)malloc(5*sizeof(int));
    for(i=0;i<5;i++)
        scanf("%d",p+i); /* 10,20,30,40,50 */
    rp=(int *)realloc(p,10*sizeof(int));
    for(i=5;i<10;i++)
        scanf("%d",rp+i); /* 9,8,6,5,4 */
    for(i=0;i<10;i++){
        printf("%d...%d\n",rp[i],*(rp+i));
    }
    return 0;
}
```

Output▼

10. Which of the given statements about the following code snippet is/are correct?

```
void fun(){
    int *q=(int *)malloc(sizeof(int));
    *q=20;
}
int main(){
    int *p;
    int *r=NULL;
    fun();
    return 0;
}
```

- (i) p is a wild pointer
- (ii) r is a NULL pointer
- (iii) q is dangling pointer
- (iv) p is dangling pointer
- (v) fun() is making memory leak

Output▼

11. A two-dimensional array can be represented as an **array of pointers** to one-dimensional arrays. Each one-dimensional array will correspond to one row within the original two-dimensional array. Create a program to generate a two-dimensional array **dynamically** using array of pointers and display that array. Additionally, find the transpose of that two-dimensional array, determine the maximum and minimum elements of the array.

Code here▼

Code here▼

Code here▼